Achieving RGB Depth-Estimation Student Convergence on Gap Terrain: Experiment Plan

Parkour Student Distillation Project

March 2026

1 Problem Statement

A Go2 quadruped is trained to traverse parkour terrain (stairs, hurdles, gaps) via DAgger-based teacher–student distillation. The *teacher* uses ground-truth (GT) scandot heightmaps and achieves 100% success on all terrain families. Two student variants receive only front-facing depth images through a shared CNN + GRU encoder:

1. **GT depth student** — uses simulator-rendered depth. Currently at **100% success on all families** (step 2939/5000).
2. **RGB depth-estimation student** — uses Depth Anything V2 (DA-V2) inferred from RGB. Currently at **0% gap success** (step 1088), despite 100% stairs and 79% hurdles.

The gap between the two students must be closed. This document enumerates every hypothesis, proposes targeted experiments, and defines a rigorous methodology for parallel execution across 4 GPUs.

2 Background and Current Architecture

2.1 DAgger Distillation Pipeline

- **Teacher**: PPO-trained, proprioception (53) + scandots (132) → MLP $[512, 256, 128]$ → 12 joint actions.
- **Student depth encoder**: depth image $(1, 58, 87)$ → CNN → 32-dim + masked proprio (53) → MLP $[85, 128, 32]$ → GRU (512 hidden) → MLP → latent (32) + yaw (2).
- **Student actor**: proprio (53, with predicted yaw injected via MTS) + latent (32) → MLP $[512, 256, 128]$ → 12 actions.
- **Loss**: $\mathcal{L} = \|a_{\text{teacher}} - a_{\text{student}}\|_2 + \|y_{\text{oracle}} - y_{\text{pred}}\|_2$, windowed BPTT (window = 24 steps) through GRU, Adam LR $= 2 \times 10^{-3}$.

2.2 Depth Normalization Pipelines

GT depth (simulator → student):

$$d_{\text{norm}} = \frac{\text{clamp}(d_{\text{raw}},\ 0,\ 2)}{2} - 0.5 \quad \Rightarrow \quad d_{\text{norm}} \in [-0.5,\ 0.5]$$

Fixed, absolute mapping. Mean ≈ 0.41 (most pixels at far clip).

Inferred depth (DA-V2 metric outdoor → EMA normalization):

$$d_{\text{norm}} = \frac{d_{\text{raw}} - \text{EMA}_{p2}}{\text{EMA}_{p98} - \text{EMA}_{p2}} - 0.5 \quad \Rightarrow \quad d_{\text{norm}} \in [-0.5,\ 0.5]$$

Adaptive, EMA $\alpha = 0.02$ over batch percentiles. Mean ≈ 0.0.

2.3 Measured Depth Estimation Quality

Model	Raw Range	Spatial r	Direction	Affine MAE
GT depth (simulator)	0.45–2.0 m	1.00	—	0.000
DA-V2 Relative Small	0.4–5.4 (disp.)	−0.59	Inverted	0.110
DA-V2 Metric Indoor Small	1.0–4.5 m	+0.72	Correct	0.093
DA-V2 Metric Outdoor Small	4.6–21.0 m	+0.79	Correct	0.091

Table 1: Depth estimation model comparison on Genesis synthetic renders (106×60). r = Pearson correlation with GT depth per-pixel. Affine MAE after optimal least-squares alignment.

Key finding: "Metric" models are *not* metric-accurate on synthetic renders — the outdoor model predicts 5–10× too far. However, it has the best *spatial correlation* ($r = 0.79$), meaning it preserves relative depth structure.

3 Hypotheses

We identify seven hypotheses for why the RGB student fails on gaps, ordered by estimated likelihood and ease of testing:

H1 Normalization distribution mismatch. EMA normalization maps to mean ≈ 0 with uniform spread, while GT maps to mean ≈ 0.41 with heavy right skew (most pixels at far clip). The CNN+GRU may need the skewed distribution to reliably detect gaps as "outlier low" pixels.

H2 Depth estimation spatial resolution insufficient for gaps. DA-V2 at 106×60 input may not resolve the narrow gap geometry (∼5–10 pixel wide stripe at 0.5 m distance). Larger models (base/large) or higher resolution may help.

H3 Depth estimation update frequency too low. Current `update_interval=2` means the depth model runs every 2nd sensor step (every 10 physics steps = 200 ms). Gap crossing requires precise timing; stale depth may cause missed decisions.

H4 BPTT window too short for noisy depth. With noisier depth, the GRU needs more temporal context to filter signal from noise. Window = 24 may be insufficient; extreme-parkour uses similar but with cleaner GT depth.

H5 Learning rate / optimizer dynamics. The depth encoder introduces more parameters with noisier gradients. A lower LR or warmup schedule may stabilize training.

H6 Depth noise level too high. `depth_noise_level = 0.1` adds uniform noise ±0.1 to normalized depth. Combined with estimation noise, total noise may overwhelm the gap signal (gaps are ∼0.1–0.2 normalized units different from background).

H7 Depth model wrong entirely. Perhaps a different backbone (Metric3D, UniDepth, Video Depth Anything) would give better spatial structure on synthetic renders.

4 Experimental Design

4.1 Phase 0: Oracle Experiments (Critical Controls)

These experiments isolate the contribution of each component. They must run *first* because they determine which subsequent experiments are worth running.

ID	Modification	What it tests	Baseline
`O1-gt-ema`	GT depth + EMA normalization (bypass simulator linear norm, apply EMA to raw GT meters)	Is the EMA normalization itself the problem, independent of the depth model? If GT+EMA also fails on gaps, normalization is the bottleneck.	GT student
`O2-gt-affine`	GT depth + random affine perturbation: $d' = (1 + \epsilon_s)\, d + \epsilon_b$, $\epsilon_s \sim \mathcal{U}(-0.3, 0.3)$, $\epsilon_b \sim \mathcal{U}(-0.2, 0.2)$ per frame	How robust is the CNN to distribution shift? If GT+affine still works, the CNN can tolerate distribution differences.	GT student
`O3-rgb-gtfallback`	RGB student with `update_interval=1` (depth estimation every sensor step)	Is the 2× update interval causing stale depth on gaps?	Current RGB

Table 2: Phase 0 oracle experiments.

4.2 Phase 1: Normalization Sweep

Conditioned on Phase 0 results. If `O1-gt-ema` fails, normalization is the bottleneck and these are high priority.

ID	Configuration	Rationale
`N1-affine-fit`	Pre-computed global affine: $d_{\text{norm}} = 0.088 \cdot d_{\text{raw}} + 0.429 - 0.5$, then clamp to $[-0.5, 0.5]$ (from least-squares fit)	Fixed mapping avoids EMA adaptation lag. Uses the measured optimal affine alignment to GT.
`N2-ema-fast`	EMA with $\alpha = 0.1$ (5× faster adaptation)	Current $\alpha = 0.02$ may be too slow to track scene changes during curriculum progression.
`N3-ema-slow`	EMA with $\alpha = 0.005$ (4× slower)	Slower EMA = more stable normalization; tests if EMA drift is the issue.
`N4-quantile-wide`	EMA on p_1/p_{99} instead of p_2/p_{98}	Wider percentile bounds preserve more of the distribution tails, which may contain the gap signal.
`N5-running-mean-std`	Normalize as $(d - \mu_{\text{EMA}})/\sigma_{\text{EMA}}$, then tanh /2 to map to $[-0.5, 0.5]$	Mean/std normalization (like BatchNorm) instead of percentile; may better preserve gap outlier structure.

Table 3: Phase 1 normalization experiments.

4.3 Phase 2: Depth Model Sweep

If Phase 0 shows the depth model is the bottleneck (i.e., `O1-gt-ema` succeeds), these experiments test alternative depth estimation backends.

ID	Model	Rationale
`D1-indoor`	DA-V2 Metric Indoor Small ($r = 0.72$, range 1.0–4.5 m)	Closer absolute range to GT than outdoor; may work better with fixed affine normalization.
`D2-relative-inv`	DA-V2 Relative Small with inversion ($d' = d_{\max} - d$) + EMA norm	Relative model has inverted output; after inversion, $r \approx +0.59$. Tests if ordinal depth with correct direction suffices.
`D3-outdoor-base`	DA-V2 Metric Outdoor **Base** (r likely > 0.79)	Larger encoder may give better spatial resolution for gap detection. Slower but more accurate.
`D4-video-metric`	Video Depth Anything Metric Small (temporal consistency built-in)	Uses temporal priors across frames; may give smoother depth for the GRU. Note: streaming per-env state, higher latency.

Table 4: Phase 2 depth model experiments.

4.4 Phase 3: Training Hyperparameter Sweep

If the depth signal is adequate (Phases 0–2 confirm), these experiments tune the learning dynamics.

4.5 Phase 4: Architecture Modifications

If training dynamics are not the issue, these experiments modify the student architecture.

4.6 Phase 5: Combined Best Configuration

After Phases 0–4 identify which individual changes help, run a final experiment combining the top 2–3 improvements.

5 Possible Bugfixes to Investigate

Before running sweeps, we should audit the following potential bugs:

B1 EMA not reset on curriculum change. When the terrain curriculum advances (harder terrains), the depth distribution shifts. The EMA may lag behind, giving incorrect normalization for the new terrain. *Fix*: Reset EMA state when terrain level changes, or increase α during level transitions.

B2 Crop mismatch between GT and inferred paths. GT depth is cropped by the *simulator* before normalization. Inferred depth is cropped in `_process_inferred_depth`. Verify the crop parameters match exactly and the resulting FOVs are identical. Currently both use `crop_top=0, bottom=2, left=4, right=4` from `depth_camera_config`, which should be correct.

B3 Inferred depth buffer timing. The GT path writes to the depth buffer via the simulator's `_update_depth_images` (shift right, insert at 0). The inferred path does its own shift in `_refresh_student_depth`. Verify the timing is identical: both should shift *after* the new frame is available, and the student should see the *previous* frame (index 1).

ID	Configuration	Rationale
`T1-bptt32`	BPTT window = 32 (from 24)	Longer temporal gradient flow through GRU; helps if the depth encoder needs more context to learn gap detection from noisy input.
`T2-bptt48`	BPTT window = 48	Even longer window; tests diminishing returns vs. memory cost.
`T3-lr-low`	Learning rate = 5×10^{-4} (4× lower)	Slower, more stable optimization; may prevent the oscillation pattern where gap success spikes then collapses.
`T4-lr-warmup`	LR warmup: linear $0 \to 2 \times 10^{-3}$ over first 200 iterations	Prevents early instability while EMA normalization is still calibrating.
`T5-noise-zero`	`depth_noise_level = 0.0`	Removes additive depth noise entirely. Tests if 0.1 noise overwhelms the already-noisy inferred depth signal.
`T6-noise-low`	`depth_noise_level = 0.02`	Reduced noise: 5× lower than current 0.1.
`T7-steps240`	`num_steps_per_env = 240` (from 120)	Longer rollouts give more data per gradient step; may help with sparse gap encounters (gaps are 1/3 of terrain).
`T8-update-1`	`update_interval = 1` (depth est every sensor step)	Same as `O3` but in combination with best normalization from Phase 1.

Table 5: Phase 3 training hyperparameter experiments.

B4 EMA initialization from first batch. The first batch may be atypical (all envs start from the same initial position, seeing similar depth). This seeds the EMA with a biased estimate. *Fix*: Initialize from the mean of the first K batches instead of just the first.

B5 GRU hidden not reset on env reset for inferred path. Verify that `reset_gru(env_ids)` is called for reset envs in the inferred depth path. If GRU hidden carries stale temporal state across episode boundaries, it could confuse gap detection. (This should be handled by the runner, but worth auditing.)

B6 Depth estimation model receives wrong color format. Genesis may return RGB or BGR; DA-V2 expects RGB. Verify the color channel ordering in `_rgb_batch_to_numpy`.

6 Methodology

6.1 Evaluation Metric

The primary metric is `Episode/success_gap` averaged over the last 200 training iterations. Secondary metrics:

- `Episode/success_stairs` — must not regress below 90%.
- `Episode/success_hurdle_block` — must not regress below 60%.
- `Loss/action` — lower is better; indicates distillation quality.

ID	Modification	Rationale
`A1-proprio-hist`	Add 10-frame proprioceptive history encoder (1D conv as in extreme-parkour) as additional input to actor	Provides a non-vision temporal signal. Gap crossing requires precise timing; joint feedback history may compensate for noisy depth.
`A2-gru1024`	GRU hidden dim = 1024 (from 512)	Larger temporal memory. The GRU must encode both depth features and temporal state; 512 may be insufficient with noisy input.
`A3-2frame-input`	Pass both current *and* previous depth frame as 2-channel input to CNN: shape $(2, 58, 87)$	Explicit temporal stacking at CNN level, bypassing GRU for motion cues. Requires modifying `DepthBackbone58x87` input channels.

Table 6: Phase 4 architecture experiments.

- `Episode/progress_gap` — gap progress ratio, useful when success is 0% (partial credit).

6.2 Success Criteria

An experiment is considered **successful** if it achieves:

- `success_gap` $\geq 50\%$ sustained for ≥ 200 iterations, OR
- `success_gap` $\geq 30\%$ with a clear upward trend at step 2000.

The ultimate target is `success_gap` $\geq 80\%$, matching the GT student's trajectory.

6.3 Training Protocol

- Each experiment runs for **2000 iterations** (sufficient to see gap learning based on GT student trajectory).
- `num_envs = 48`, `num_steps_per_env = 120` (unless varied by the experiment).
- Terrain curriculum enabled (default), all families mixed.
- Seed = 1 (default). If a promising result is found, confirm with seed = 42.
- Training launched via:
  `CUDA_VISIBLE_DEVICES=$GPU .venv/bin/python legged_gym/scripts/train.py --task go2_parkour_depth_est_student --headless --max_iterations 2000`
- Config overrides applied programmatically before launch (modify config class attributes in the training script).

6.4 GPU Allocation

Four GPUs are available. Experiments are scheduled in phases with dependencies:

6.5 Implementation Notes for Config Overrides

Each experiment requires modifying the config before training. The cleanest approach is a wrapper script that:

1. Imports the config classes.
2. Applies experiment-specific overrides (e.g., changing EMA alpha, model type, BPTT window).
3. Calls the standard training entry point.

Phase	GPU 0	GPU 1	GPU 2	GPU 3
0	O1-gt-ema	O2-gt-affine	O3-rgb-gtfallback	(continue current run)
Wait for Phase 0 results to determine priority of Phase 1 vs 2				
1	N1-affine-fit	N2-ema-fast	N5-running-meanstd	T5-noise-zero
1b	N3-ema-slow	N4-quantile-wide	T1-bptt32	T3-lr-low
2	D1-indoor	D2-relative-inv	D3-outdoor-base	D4-video-metric
3	T2-bptt48	T4-lr-warmup	T6-noise-low	T8-update-1
4	A1-proprio-hist	A2-gru1024	A3-2frame-input	(best combo)

Table 7: GPU scheduling plan. Each cell runs for ∼2000 iterations. Phase 0 takes priority; subsequent phases are re-ordered based on results.

For overrides that require code changes (e.g., affine normalization, mean/std normalization, proprio history encoder), a git worktree per experiment keeps changes isolated.

6.6 Monitoring and Early Stopping

A monitoring script reads TensorBoard events every 5 minutes and reports:

- Current step, `success_gap`, `progress_gap`, `action_loss` for each running experiment.
- Early stop if `success_stairs` drops below 50% for 100 consecutive iterations (training has diverged).
- Highlight any experiment where `success_gap` $> 20\%$ (promising signal).

7 Execution Priority

Based on the evidence so far, the recommended execution order is:

1. **Phase 0 (oracle)** — *Must run first.* `O1-gt-ema` is the single most informative experiment. If GT depth + EMA normalization works, the problem is purely depth model quality. If it fails, the problem is normalization.
2. **Bugfix audit (B1–B6)** — Quick code review, no GPU needed. Do in parallel with Phase 0.
3. **Phase 1 or 2** — Conditioned on Phase 0:
   - If `O1-gt-ema` **fails**: normalization is the bottleneck → prioritize Phase 1.
   - If `O1-gt-ema` **succeeds**: depth model is the bottleneck → prioritize Phase 2.
4. **Phase 3** — Training hyperparameters. Run the most promising 4 in parallel after identifying the best normalization/model.
5. **Phase 4** — Architecture changes. Only if Phases 1–3 are insufficient.
6. **Phase 5** — Combine the top 2–3 improvements from all phases into a single run.

8 Risk Assessment

- **High risk**: The fundamental limit may be DA-V2's spatial resolution on 106×60 synthetic images. Gap features are ∼5–10 pixels wide at decision distance. If no depth model resolves gaps, we may need to increase camera resolution (but this slows training).

- **Medium risk**: The oscillation pattern (gap success spikes then collapses) suggests a training stability issue, not a fundamental information deficit. This is encouraging — the student *briefly* learns gaps, then forgets. LR scheduling or longer BPTT may suffice.
- **Low risk**: The curriculum may systematically under-sample gap terrain, giving the optimizer too few gap gradients. Could address with curriculum weighting, but this is unlikely given the teacher converges fine.

9 Expected Timeline

- **Phase 0**: 2000 iterations ≈ 2–3 hours per run, 3 runs in parallel.
- **Phase 1–2**: 4 runs in parallel, 2–3 hours each; two sub-rounds = 4–6 hours.
- **Phase 3**: 4 runs in parallel, 2–3 hours.
- **Phase 4**: 3 runs, 2–3 hours.
- **Phase 5**: 1 final run, 5000 iterations ≈ 5–6 hours.
- **Total**: ∼12–18 hours of wall time with 4 GPUs.

10 Appendix: Experiment Implementation Details

10.1 `O1-gt-ema`: GT Depth with EMA Normalization

Override the GT student config to use EMA normalization instead of simulator linear normalization. This requires:

1. Set `add_depth = True`, `add_rgb = False`.
2. Enable the EMA normalization path in `_process_inferred_depth` for GT depth.
3. Concretely: force `_use_inferred_depth = True` but provide simulator GT depth (raw meters) as the "inferred" source. This reuses the existing EMA pipeline on GT data.
4. Alternative implementation: add a `force_ema_normalization` flag that applies EMA to the simulator's raw depth output *before* its linear normalization.

10.2 `O2-gt-affine`: GT Depth with Random Affine Perturbation

After the simulator's linear normalization, apply a per-batch random affine:

$$d' = (1 + \epsilon_s) \cdot d + \epsilon_b$$
$$\epsilon_s \sim \mathcal{U}(-0.3, 0.3), \quad \epsilon_b \sim \mathcal{U}(-0.2, 0.2)$$

Clamp result to $[-0.5, 0.5]$. This tests CNN robustness to distribution shift.

10.3 `N1-affine-fit`: Pre-computed Affine Normalization

Replace EMA normalization with a fixed affine derived from least-squares calibration:

$$d_{\text{GT}} \approx 0.088 \cdot d_{\text{inferred}} + 0.858$$

Then apply standard GT normalization: $d_{\text{norm}} = \text{clamp}(d_{\text{aligned}}, 0, 2)/2 - 0.5$.

10.4 `N5-running-meanstd`: Mean/Std Normalization

Replace percentile-based EMA with running mean and standard deviation:

$$d_{\text{norm}} = \tanh\left(\frac{d - \mu_{\text{EMA}}}{2\,\sigma_{\text{EMA}}}\right) \cdot 0.5$$

This preserves outlier structure (gaps produce values far from the mean) while bounding the output to $[-0.5, 0.5]$.

10.5 `D2-relative-inv`: Inverted Relative Model

Use the relative DA-V2 model and invert the output: $d' = \max(d_{\text{batch}}) - d_{\text{raw}}$, then apply EMA normalization. This corrects the inverted direction ($r = -0.59 \rightarrow +0.59$).

10.6 `A1-proprio-hist`: Proprioceptive History Encoder

Add a 1D convolutional encoder over the last 10 proprioceptive frames (following extreme-parkour [**?**]):

- Buffer: $(B, 10, 53)$ proprioceptive observations.
- 1D Conv: $53 \rightarrow 32$ channels, kernel 4, stride 2 $\rightarrow$ flatten $\rightarrow$ MLP $\rightarrow$ 32-dim embedding.
- Concatenate with depth latent before the actor MLP.

10.7 `A3-2frame-input`: Two-Channel Depth Input

Modify `DepthBackbone58x87` to accept 2-channel input: `nn.Conv2d(2, 32, ...)` instead of `nn.Conv2d(1, 32, ...)`. Pass both `_depth_buffer[:, 0]` (current) and `_depth_buffer[:, 1]` (previous) as a stacked tensor. This gives the CNN explicit access to temporal differences without relying on the GRU.